

Frontend Assignment-II *(thoery)*

Jagriti Singh

NC-19

22515000056

QUE-1) What is CSS Box Model?

Answer- The **CSS Box Model** is a concept used to describe how every HTML element is represented as a rectangular box on a webpage. It consists of **four parts: Content, Padding, Border, and Margin.**

- **Content:** The actual text or image inside the element.
- **Padding:** Space between the content and the border.
- **Border:** A line surrounding the padding and content.
- **Margin:** Space outside the border that separates the element from other elements.

Thus, the CSS Box Model helps control the **size, spacing, and layout** of HTML elements.

QUE-2) What is media query? Explain with a sample code.

Answer- A **media query** is a CSS technique used to apply different styles to a webpage based on the **screen size, device type, or other characteristics**. It is mainly used to make websites **responsive**, so that they look good on mobile phones, tablets, and computers.

EXAMPLE: In CSS-

```
body {  
    background-color: lightblue;  
}
```

```
@media screen and (max-width: 600px) {  
    body {
```

```
background-color: lightgreen;
}
}
```

Here, the background is **light blue** on screens wider than 600px. When the screen width is **600px or less**, the background changes to **light green**.

QUE-3) What are the features of Bootstrap?

Answer- **Bootstrap** is a popular **front-end framework** used to create responsive and attractive websites quickly. It provides pre-designed CSS classes and components.

Main features of Bootstrap are:

1. **Responsive Design** – Websites automatically adjust to different screen sizes.
2. **Mobile-First Approach** – Designed to work well on mobile devices.
3. **Grid System** – Provides a flexible 12-column grid for creating layouts.

QUE-4) Write a example of basic grid structure in Bootstrap?

Answer- Bootstrap provides a **12-column grid system** that is used to create responsive webpage layouts. The basic grid structure consists of a **container, row, and columns**.

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
<title>Bootstrap Grid</title>
```

```
<link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css"
rel="stylesheet">
```

```
</head>
```

```
<body>
```

```
<div class="container">

  <div class="row">

    <div class="col-4">Column 1</div>

    <div class="col-4">Column 2</div>

    <div class="col-4">Column 3</div>

  </div>

</div>

</body>

</html>
```

Explanation:

- container provides a fixed-width or responsive container.
- row creates a horizontal row.
- col-4 creates a column that occupies **4 out of 12 columns**.
- Three col-4 columns make a complete row: **4 + 4 + 4 = 12**.

Thus, Bootstrap's grid system helps create **organized and responsive layouts**.

QUE-5) What is CSS Selectors?

Answer- **CSS selectors** are patterns used to **select HTML elements** that we want to style using CSS. They tell the browser which HTML elements should receive a particular style.

Common types of CSS selectors are:

1. **Element Selector** – Selects elements by their tag name.

```
p{

  color: blue;

}
```

Class Selector – Selects elements using a class name.

```
.heading{  
    color: red;  
}
```

ID Selector – Selects an element using its ID.

```
#title {  
    color: green;  
}
```

Universal Selector – Selects all HTML elements.

```
* {  
    margin: 0;  
}
```

Group Selector – Applies the same style to multiple elements.

```
h1, h2, p {  
    color: purple;  
}
```

Thus, CSS selectors are used to **target specific HTML elements and apply styling to them.**

QUE-6) What are CSS backgrounds, list the properties?

Answer- **CSS backgrounds** are used to set and control the **background appearance of HTML elements**. We can use CSS backgrounds to add colors, images, and control their position, size, and repetition.

CSS Background Properties:

1. **background-color** – Sets the background color.
2. **background-image** – Sets an image as the background.
3. **background-repeat** – Controls whether the background image repeats.

QUE-7) How can we overlap elements in CSS?

Answer- Elements can be **overlapped in CSS** by using the **position** property along with the **z-index** property. Positioning allows elements to be placed at specific locations, while **z-index** controls which element appears in front of another.

QUE-8) What are the different positioning properties in CSS?

Answer- The **position property in CSS** is used to control how an HTML element is placed on a webpage. There are five main positioning properties:

1. **Static** – This is the default position. The element remains in the normal flow of the webpage.
2. **Relative** – The element is positioned relative to its original or normal position.
3. **Absolute** – The element is positioned relative to its nearest positioned parent element.
4. **Fixed** – The element remains in a fixed position relative to the browser window, even when the page is scrolled.
5. **Sticky** – The element behaves normally until a certain scroll position is reached, after which it stays fixed within its containing area.

Thus, CSS positioning properties are used to **control the placement and movement of elements on a webpage**

QUE-9) What is ECMAScript?

Answer- **9. What is ECMAScript?**

ECMAScript is a **standard or specification for scripting languages**. It defines the rules, syntax, and features that scripting languages should follow.

JavaScript is the most widely used implementation of the ECMAScript standard.

ECMAScript is maintained and standardized by **ECMA International**.

Different versions of ECMAScript have introduced new features, such as **let, const, arrow functions, classes, modules, and promises**.

Thus, **ECMAScript provides the standard on which modern JavaScript is based**.

QUE-10) What is the difference between var, let and const in JavaScript. Explain Temporal Dead zone?

Answer- **10. Difference Between var, let and const & Temporal Dead Zone**

var, let, and const are used to declare variables in JavaScript.

- **var:** Function-scoped, can be redeclared and reassigned.
- **let:** Block-scoped, can be reassigned but cannot be redeclared in the same scope.
- **const:** Block-scoped, cannot be redeclared or reassigned.

Temporal Dead Zone (TDZ):

The TDZ is the time between entering a scope and declaring a let or const variable. During this period, the variable cannot be accessed and accessing it causes a **ReferenceError**.